Python Engineering

WEEK 03 - Modules & Coordinates | CLASS 09 - Modules & Imports | EX17

Preview: we will officially start this exercise after EX16b is complete. EX17 opens a new week and a new class.

Folder path

```
01_python_bootcamp/
`-- week_03_modules_and_coordinates/
    `-- class_09_modules_imports/
        |-- notes/
        |-- exercises/
        |   `-- ex17_vertical_motion_modules/
        |       |-- main.py
        |       |-- vertical_motion.py
        |       `-- data_processing.py
        `-- mini_project/
```

Goal

Stop keeping the entire program in one file. We will separate simulation, data processing, and the main program into our own modules and learn how to import them.

1. vertical_motion.py

```
validate_inputs(...)
euler_step(...)
simulate_vertical_motion(...)
analytical_solution(...)
calculate_errors(...)
```

2. data_processing.py

Create a simple function receiving trajectory_array and config and returning the data-processing results already developed in EX16.

```
def process_trajectory(trajectory_array, config):
    ...
    return processing_results, processed_data
```

3. main.py

main.py becomes the coordinator: it defines config, calls the modules, and displays results. Euler equations and filtering details should live outside main.py.

```
from vertical_motion import (
    validate_inputs,
    simulate_vertical_motion,
    analytical_solution,
    calculate_errors
)

from data_processing import process_trajectory
```

4. What does import mean?

Syntax	What it does	Usage
import module	imports the whole module	module.function()
import module as alias	imports with a short name	np.array(...)
from module import function	imports a specific name	function(...)

5. __name__ == '__main__'

We introduce the main guard. It lets a file behave differently when executed directly versus imported as a module.

```
def main():
    ...

if __name__ == '__main__':
    main()
```

When main.py is executed directly, the condition is True. When another file imports it, this block does not run automatically.

6. Your EX17 mission

1. Create the three files.
2. Move existing functions without changing their mathematics.
3. Import simulation functions into main.py.
4. Create process_trajectory() in data_processing.py.
5. Import and call process_trajectory() from main.py.
6. Keep config in main.py.
7. Add a main() function.
8. Add if __name__ == '__main__': main().
9. Run main.py from the terminal.
10. Verify results match EX16.

7. Exercise rule

We do not change the physics or numerical method. The learning goal is architectural: **separate responsibilities and reuse code across files.**

Completion criterion

EX17 is complete when main.py can execute the full simulation using code imported from your own modules and you can explain the difference between importing a whole module and importing one specific function.